N - Handling number registers.

Syntax:

```
N(regname)OPERATIONS
```

Examples:

```
n(x):#
n(value)+2*4
n(octal)b8:8#100
$n(buflen)A
na<12 jf jm/Register "a" is less than 12/
```

Options:

`(regname)`
: is the name of a number register. Number register names follow the same rules as buffer names.

`OPERATION`
: is one of the many operations described below. In all these operations, `M` stands for any integer, positive or negative.

`:M`
: Assigns Register `(regname)` the value `M`.

FRED supports the usual arithmetic operations. These can be combined in strings as in

```
n(xx)+1*5
```

Computation proceeds from left to right.

`-`
: Switches the sign of the value in `(regname)`. For example, after

  ```
  n(X):4 n(X)-
  ```

  the value in `X` will be `-4`.

`|`
: Changes the value in `(regname)` to its absolute value. For example, after

  ```
  n(X):-4 n(X)|
  ```

  the value in `X` will be `4`.

`+M`
: Adds `M` to `(regname)`.

`-M`
: Subtracts `M` from `(regname)`.

`*M`
: Multiplies `(regname)` by `M`.

`/M`
: Divides `(regname)` by `M` (integer divide).

`%M`
: `(regname)` becomes `(regname)` modulo `M`.

These operations work with the bits of the integer value in `(regname)`.

`~`

Complements the integer value -- all 1-bits are changed to 0-bits, and all 0-bits are changed to 1-bits.

`|M`

Inclusive OR. The new value of `(regname)` will have 1-bits wherever `M` or the old value (or both) had 1-bits; other bit locations will contain 0-bits.

`^M`

Exclusive OR. The new value of `(regname)` will have 0-bits wherever both `M` and the old value had 0-bits or both had 1-bits; other bit locations will contain 1-bits.

`&M`

Bitwise AND. The new value of `(regname)` will have 1-bits wherever both `M` and the old value had 1-bits; other bit locations will contain 0-bits.

`}M`

Logical Right shift by `M` positions. Vacated bit positions will be filled with 0-bits.

`{M`

Logical Left shift by `M` bits. Vacated bit positions will be filled with 0-bits.

These operations set the condition register TRUE if the given relation is true. Otherwise, the condition register is set FALSE.

`>M`

Tests if `(regname)` is greater than `M`.

`<M`

Tests if `(regname)` is less than `M`.

`=M`

Tests if `(regname)` is equal to `M`.

`BM`

Indicates that register `(regname)` should be represented in base `M` when used in a `\N` construction. For example, `n(x)b8` indicates that register `x` should be displayed as an octal number. If the base `M` is negative, the register value will be taken to be SIGNED; otherwise, it will be taken to be UNSIGNED. Thus `B10` is used to set up an unsigned decimal register, while `B-10` is used to set up a signed decimal register (the default). See below for more on signed and unsigned values.

When working with registers in different bases, it is useful to use extended integers. See the section on Extended Integers for more details.

`DM`

This sets the minimum number of digits to be displayed when `(regname)` is used in a `\N` construction. If the value in register `(regname)` has fewer digits than the number specified, it will be padded with leading zeroes (unless a different fill character is specified as discussed below). For example,

```
n(xx):12 n(xx)d4
```

would result in `\N(xx)` being expanded as `0012`. When a number register is first allocated, the minimum number of display digits is taken to be 1.

`Fc`

When a number register has been given a wide format using the `DM` option, FRED's default is to pad to the correct width using leading zeroes. The `Fc` option tells FRED to pad with the character *c* instead. For example, `n(xx)f*` tells FRED to pad `\N(xx)` with asterisks instead of zeroes. If the register's value is negative, the minus sign will be printed adjacent to the main part of the number (*after* the fill characters instead of before).

`P`

The contents of register `(regname)` are printed on the terminal.

`A`

The value of the line address immediately preceding the `N` is assigned to register `(regname)`. For example,

```
/XYZ/n(xx)a
```

gives N(xx) the line number of the next line that contains XYZ. If only one address is specified, this address is made the current line. If more than one address is specified, "." is set to the first address in the list but the value of the register is set to the last. For example,

```
1,/XYZ/n(xx)a
```

moves to the first line of the buffer but sets the value of N(xx) to the next line that contains XYZ. If no line address is specified, "." remains unchanged and its value is assigned to (regname).

L
: assigns (regname) the number of characters in one or more lines. If no address is specified, (regname) gets the number of characters in the current line. If one address is specified, (regname) gets the number of characters in that line. If two addresses are specified, (regname) gets the number of characters in all the lines between and including the given lines. In all these lengths, the new-line characters that end a line count as a character.

N manipulates the integer values contained in number registers. The number stored in register (regname) can be obtained using the stream directive \N(regname).

After any N command, the count register is set to the value in register (regname). The condition register is set by the relational N operations. FRED sets "." to the line address immediately preceding the N if such an address is specified; otherwise "." is unaffected.

In an N command, anywhere that you can specify an integer M, you can use one of the following constructs.

#
: representing the value in the count register;

$
: representing the line number of the last line in the buffer;

.
: representing the line address of the current line;

n(regname)
: representing the value in number register (regname). For example,

```
n(X)+n(Y)
```

adds the value of register Y to register X.

Be careful using the N(regname) construct and remember that all expressions are evaluated strictly left to right. For example,

```
n(X):1+n(X)
```

assigns the value 1 to X and then adds the register's value to itself. The result is 2. (Those who don't think about the left-to-right order of evaluation may think that the above expression increments X by 1.) As another example of how this construct can be tricky, consider

```
n(X)*-n(Y)
```

You might think that this multiplies X by negative Y. However, the command will be given a syntax error. After the *, FRED expects a number; instead, it finds another operator (-).

As a final example, consider the following.

```
n(X)b8 n(X):8#100
n(Y):n(X)
n(Z):\N(X)
```

The first line sets up X as an octal register and assigns it an octal value of 100. The second line assigns the value of X to Y; Y is a decimal register (by default) and will therefore contain the decimal value 64 (equivalent

to octal 100). In evaluating the third line, FRED expands `\N(X)` first. The expansion uses the octal representation of the value of `X`, so the command becomes

```
n(Z):100
```

and `Z` is assigned the DECIMAL value 100. Thus you should be careful to distinguish between `N(X)` (which is the true value of `X` when it is used in an `N` command) and `\N(X)` (which is the value of `X` written out according to its specified base).

Arithmetic operations may be combined with each other as mentioned above. If you have any `B`, `D`, or `F` operations, they must come before any other operations. A single `N` command can only contain one assignment (`:`), and if this is present, it must be the first thing that appears after the `B`, `D`, or `F` operations (if any).

A relational operation may be combined with other operations, provided it is the last operation in the string as in

```
n(xx):#>0
```

If there is a `P` operation, it must always be the last operation in the string of operations. FRED will assume a new command starts after the `P`. For example,

```
n(xx):5p+2
```

is interpreted as the two commands

```
n(xx):5p
+2
```

(so the `+2` is taken as a relative line address).

As noted above, you can use the `B` operation to "declare" a register signed or unsigned. If a register is unsigned, all operations will be carried out using unsigned arithmetic, even if argument values in the operation string contain signs. For example, consider

```
n(x)b8:100
n(x)/-1
```

The first `N` command declares `X` to be an unsigned octal register. The second divides the value in `X` by `-1`. To do the division, the `-1` is first converted to an unsigned value, and this happens to be the highest unsigned value on the computer. Doing integer division with this high unsigned value will have a result of 0 for every unsigned integer value except itself.

The difference between signed and unsigned arithmetic affects division, modulus, and comparison operations.